UTS PRAKTIKUM

Communication Protocol

Case A - REST API JSON

API Request | Data Encoding | Packet Capture | Python Parsing

Kelompok 02 - Sains Data Reguler

01 **Zahir Ali Izzaturahman**
API Tester & Postman Collection

02 **Stephanus Teo**
Packet Capture & Troubleshooting

03 **Enrico Lazuardi**
Data Format Analysis & Python Parsing

04 **Leroy Christopher Gerson**
Report, PPT, GitHub & Backup Presenter

HTTP :8088 | 4 Postman requests | 64 captured packets

9	12	2 x 8
evidence screenshots	report pages	parsed DataFrame

Repository evidence profile | 9 June 2026

Case, tujuan praktikum, dan alur request

REST API lokal sebagai jalur komunikasi client-server yang dapat diamati dan direplikasi.

CASE A - REST API JSON

- Uji 2 GET dan 2 POST melalui Postman
- Amati HTTP request-response dan JSON
- Tangkap traffic lokal dengan Wireshark
- Parse /api/transactions ke DataFrame dan CSV

TUJUAN

Membuktikan seluruh pipeline komunikasi:

client -> protocol -> server -> format data -> analisis tabular

5
endpoint digunakan

4
Postman requests

4
HTTP exchanges

1
CSV output

BASE URL

http://127.0.0.1:8088

ALUR REQUEST

Postman / Python
client

->

HTTP GET / POST
protocol

->

Mock API :8088
server

->

JSON response
encoding

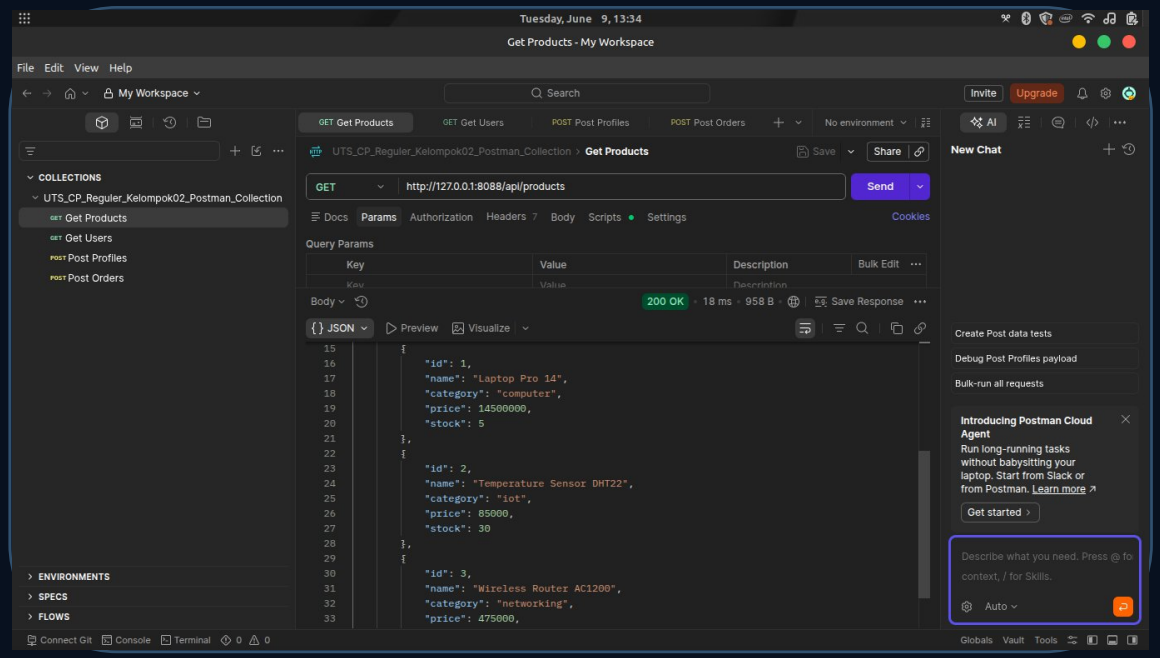
->

Wireshark / pandas
evidence

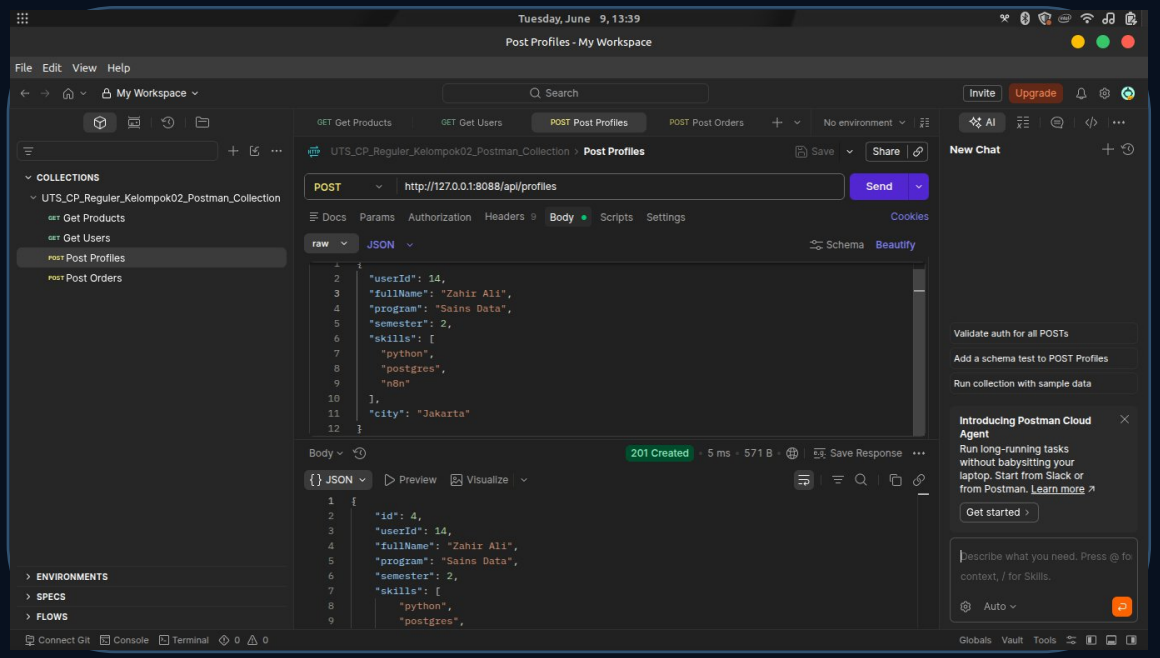
Evidence request GET/POST dan hasil response

Empat request Postman menghasilkan status sesuai semantik HTTP: GET 200, POST 201.

GET /api/products | 200 OK



POST /api/profiles | 201 Created



GET **/api/products**

200 OK

GET **/api/users**

200 OK

POST **/api/profiles**

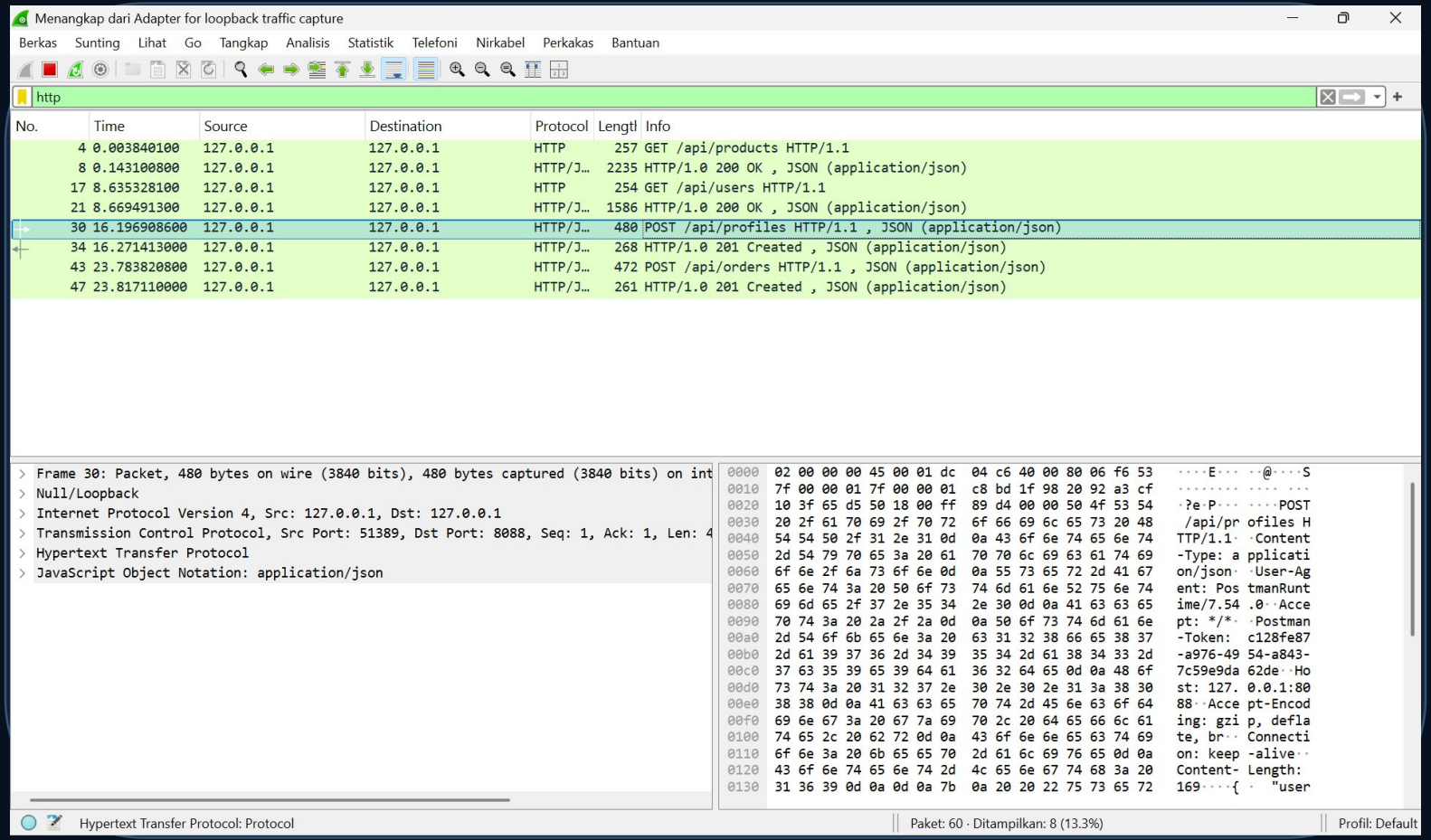
201 Created

POST **/api/orders**

201 Created

Evidence packet capture dan analisis paket penting

PCAP diverifikasi dari adapter loopback: HTTP terlihat karena endpoint menggunakan http, bukan https.



64
total packets

198.48 s
capture duration

8
HTTP messages

4
request-response pairs

PAKET PENTING

- Src dan dst: 127.0.0.1
- TCP destination port: 8088
- POST Profiles frame: 480 B
- Payload JSON Content-Length: 169
- Response: HTTP/1.0 201 Created
- Content-Type: application/json

Analisis: loopback memudahkan observasi; pada HTTPS payload JSON akan dilindungi TLS.

Data format analysis: JSON, XML, dan Protobuf

Format aktual proyek adalah JSON; XML dan Protobuf dibandingkan sebagai alternatif pertukaran data.

JSON - DIPILIH

Representasi: text key-value

Kelebihan:
- mudah dibaca
- native di REST API
- langsung diparse Python

Trade-off:
- lebih verbose dari binary
- schema tidak wajib

XML

Representasi: text bertag

Kelebihan:
- struktur eksplisit
- validasi schema kuat

Trade-off:
- tag membuat payload besar
- parsing lebih kompleks

PROTOBUF

Representasi: binary

Kelebihan:
- payload ringkas
- serialisasi cepat

Trade-off:
- perlu .proto/schema
- tidak human-readable

Evidence aktual: Content-Type application/json; response berbentuk object dengan key metadata + array data.

Python parsing dan output tabel / CSV / DataFrame

requests mengambil JSON, pandas.json_normalize membentuk tabel, lalu hasil disimpan sebagai CSV.

PARSING SCRIPT

```
url = "http://127.0.0.1:8088/api/transactions"
response = requests.get(url)
response.raise_for_status()
data = response.json()

records = data["data"]
df = pd.json_normalize(records)
df.to_csv("output/parsed_result.csv", index=False)
```

Script final juga memiliki fallback untuk list, data, transactions, results, atau dict satu baris.

```
Status Code: 200
Content-Type: application/json; charset=utf-8

Raw JSON type: <class 'dict'>
Raw JSON preview:
dict_keys(['resource', 'case', 'count', 'methods', 'canonicalPath', 'itemPathExample', 'data'])

Preview DataFrame:
   id transactionId  ...                    paidAt          channel
0   1  TRX-2026-001  ...  2026-06-01T09:00:00+0700  payment-gateway
1   2  TRX-2026-002  ...                      None  payment-gateway

[2 rows x 8 columns]

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   id             2 non-null      int64
 1   transactionId  2 non-null      object
 2   orderId        2 non-null      object
 3   method         2 non-null      object
 4   amount         2 non-null      int64
 5   status         2 non-null      object
 6   paidAt         1 non-null      object
 7   channel        2 non-null      object
dtypes: int64(2), object(6)
memory usage: 256.0+ bytes
None
```

200	2	8	256 B
HTTP status	rows	columns	DataFrame memory

OUTPUT

output/parsed_result.csv

Catatan Teknis, Lesson Learned, dan Kesimpulan

Hasil berhasil direplikasi; profil proyek juga menunjukkan area yang perlu diperkuat.

CATATAN TEKNIS

Capture dilakukan melalui adapter loopback

Filter http membantu memisahkan paket penting

HTTP dipakai agar request dan response dapat diamati

Parser menangani response JSON semi-terstruktur

Status code dan Content-Type berhasil divalidasi

LESSON LEARNED

HTTP menunjukkan pola request-response secara nyata

Status 200 dan 201 membuktikan operasi GET/POST berhasil

JSON mudah diparse menjadi DataFrame dan CSV

Packet capture membuktikan komunikasi pada level jaringan

Evidence terstruktur membuat hasil praktikum mudah diaudit

KESIMPULAN

Case A REST API JSON berhasil diuji end-to-end

Postman membuktikan request GET dan POST berjalan sesuai fungsi

Wireshark membuktikan traffic HTTP terjadi pada jaringan lokal

Python berhasil mengubah response JSON menjadi data tabular

Repository GitHub menyatukan seluruh evidence agar mudah direplikasi

Kesimpulan utama: seluruh evidence membuktikan REST API JSON berhasil diuji dari tiga sisi: request-response, packet capture, dan parsing data.

Kontribusi anggota, evidence map, dan demo singkat

Slide opsional: gunakan sebaga

ROLE 1 **Zahir Ali Izzaturahman**

Postman collection + 4 request screenshots

ROLE 2 **Stephanus Teo**

PCAP + 4 packet analysis screenshots

ROLE 3 **Enrico Lazuardi**

Python parser + CSV + DataFrame evidence

ROLE 4 **Leroy Christopher Gerson**

Report + PPT + repository structure

DEMO 60 DETIK

1 **Postman** kirim GET/POST -> 2 **Wireshark** filter http -> 3 **Python** jalankan parser -> 4 **CSV** cek 2 x 8 output -> 5 **GitHub** tunjukkan evidence

Repository: github.com/LeroyChris/UTS-Communication-Protocol-Kelompok-02